Plan de gouvernance des données et de l’IA

Luxury Stock AI, dossier de certification Architecte en Intelligence Artificielle

Version 1.1, 15 octobre 2026

Table des matières

1. Objet et périmètre
2. Données et provenance
3. Principes de gouvernance
4. Dictionnaire et qualité
5. Rôles et responsabilités
6. Architecture et accès
7. Risques IA et contrôles
8. RGPD et conformité
9. Incidents et violations
10. Supervision et amélioration continue
11. Limites et références

1. Objet et périmètre

Luxury Stock AI est un système d'aide à la décision pour une marque fictive de maroquinerie de luxe. Il utilise des données synthétiques et agrégées de catalogue, ventes et stocks afin de prévoir la demande et d'identifier les risques de rupture ou de surstock.

Le périmètre couvre la collecte, la validation, la transformation, le stockage, la modélisation, l'évaluation, la restitution et la supervision. La solution ne prend pas automatiquement de décision d'achat et ne traite aucune donnée client réelle.

2. Données et provenance

Les fichiers sont synthétiques et générés par `scripts/generate_data.py` avec une seed documentée. Les sources sont :

- `products.csv`, catalogue produit (36 produits) ;
- `calendar.csv`, calendrier et split temporel (116 semaines : 104 d'entraînement, 12 de test) ;
- `sales_weekly.csv`, ventes agrégées par produit et semaine ;
- `inventory_weekly.csv`, niveaux de stock et indicateurs ;
- `forecast_horizon.csv`, période future à prévoir.

Aucun nom, email, identifiant client, adresse, donnée biométrique ou donnée de paiement n'est utilisé. Si le projet devait intégrer des données réelles, une analyse complémentaire serait nécessaire avant traitement.

Lignage

Le schéma ci-dessous montre le trajet d'une donnée, de la source à la décision. Chaque flèche porte le nom du fichier ou de la table échangé ; les couleurs correspondent aux familles de la légende.

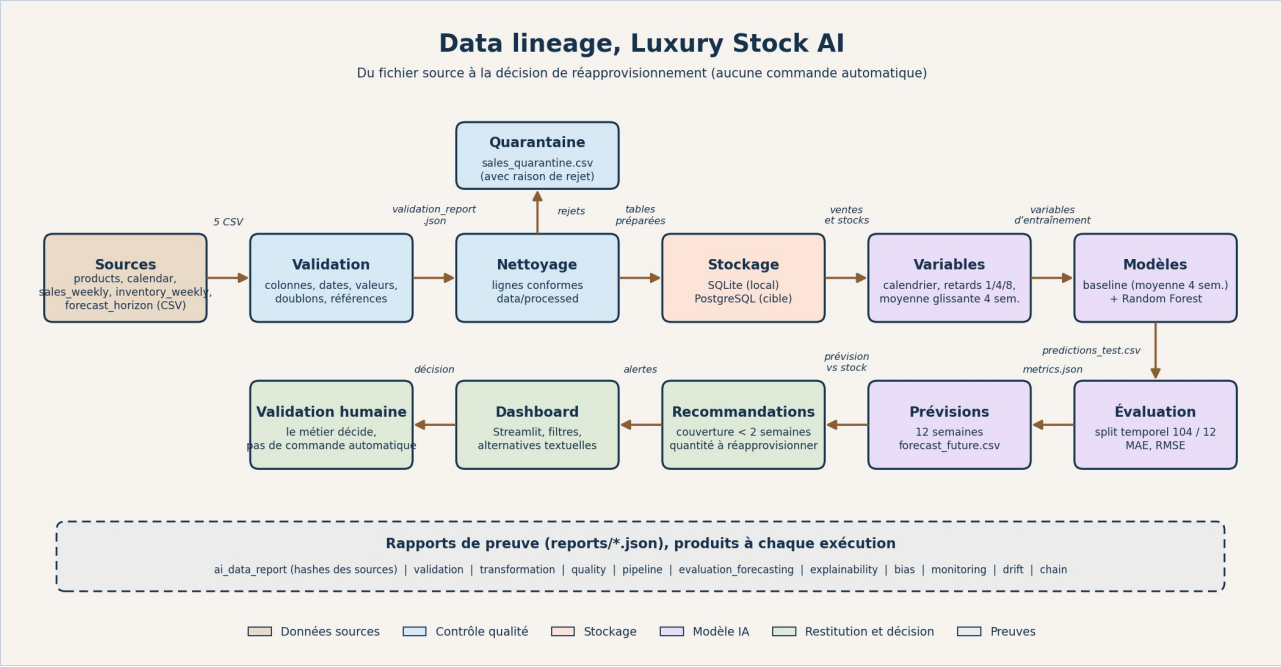


Schéma de lignage. Les cinq fichiers CSV sources passent par la validation qualité, puis le nettoyage, qui exporte les lignes rejetées dans une quarantaine. Les données propres sont stockées dans SQLite en local ou PostgreSQL en cible, puis transformées en variables temporelles et retardées. Une baseline et un modèle Random Forest sont évalués sur un split temporel de 104 semaines d'entraînement et 12 de test. Les prévisions sur 12 semaines alimentent les recommandations de stock, affichées dans le dashboard Streamlit, où le métier valide ou non chaque décision. Les rapports JSON du dossier reports servent de preuve à chaque étape.

3. Principes de gouvernance

- **Finalité** : améliorer la planification du stock et prioriser les analyses.
- **Minimisation** : ne conserver que les variables nécessaires à la prévision.

- **Qualité** : contrôler les types, dates, valeurs, références et doublons.
- **Traçabilité** : conserver la provenance, les règles de transformation et les lignes rejetées.
- **Sécurité** : limiter les accès, ne pas versionner de secret et séparer les environnements.
- **Explicabilité** : documenter les variables, la baseline, les métriques et la règle de réapprovisionnement.
- **Validation humaine** : aucune recommandation ne devient automatiquement une commande.
- **Accessibilité** : concevoir le dashboard avec des libellés lisibles, des contrastes suffisants et une alternative textuelle aux graphiques.

4. Dictionnaire et qualité

Le dictionnaire `data_dictionary.xlsx` décrit chacune des 46 colonnes des 5 tables : type, description, finalité, nullabilité, contrainte SQL, règle de qualité, exemple et caractère personnel. Le schéma `001_schema.sql` fait foi ; le test `tests/test_data_dictionary.py` échoue si une colonne manque ou est en trop.

Les contrôles du pipeline portent sur :

- la présence des colonnes obligatoires ;
- le parsing des dates ;
- les valeurs numériques manquantes ;
- les quantités négatives ;
- les références produit inconnues ;
- les doublons sur `(week_start, product_id)` ;
- la cohérence entre périodes d'entraînement et de test.

Après chargement dans PostgreSQL, `003_quality_checks.sql` ajoute six contrôles critiques (calendrier consécutif et sur des lundis, absence de fuite temporelle, grilles produits x semaines complètes, ventes rattachées à un stock) et six avertissements métier (ventes nettes, bilan de stock, cohérence ventes/stocks, ventes avant lancement, chiffre d'affaires, semaine ISO). Un contrôle critique en échec interrompt la création de la base.

Politique de traitement des anomalies

Les lignes non conformes sont toujours exportées dans `data/processed/sales_quarantine.csv` avec une raison de rejet ; elles ne sont pas supprimées silencieusement. Lorsqu'une règle `fail_on_*` de `config.yaml` est activée et qu'une ligne correspondante est rejetée, la chaîne s'arrête ensuite avec un code de retour non nul. La reprise est validée par le propriétaire métier (section 9).

5. Rôles et responsabilités

Rôle	Responsabilités
Propriétaire métier	Définit les besoins, valide les règles de stock, accepte les limites et les risques résiduels moyens.
Data owner	Garantit la définition, la provenance et la qualité des données sources ; tient le dictionnaire à jour.
Data engineer	Développe les contrôles, transformations, chargements et logs.
ML engineer	Entraîne, évalue, versionne et supervise le modèle.
Responsable sécurité	Revoit les secrets, droits, dépendances et mesures de protection.
Utilisateur métier	Interprète les alertes et garde la décision finale.
DPO ou responsable conformité	Intervient avant toute introduction de données réelles et pour l'exercice des droits.

6. Architecture et accès

Les données brutes sont en lecture contrôlée. Dans la configuration fournie, le pipeline et le dashboard utilisent SQLite (`database/luxury_stock_pipeline.sqlite`) pour un démarrage immédiat. Le schéma PostgreSQL cible (`001` à `004`) est créé et chargé à partir de `data/raw` par Docker Compose, au premier démarrage du volume. Le modèle et les rapports sont générés par des scripts versionnés.

Les secrets sont injectés par variables d'environnement à partir d'un fichier `.env` non versionné (modèle : `.env.example`). `docker-compose.yml` ne contient aucun mot de passe et publie PostgreSQL uniquement sur `127.0.0.1`.

Les accès PostgreSQL suivent le moindre privilège (`004_roles.sql`) :

Rôle	Type	Droits	Usage
`luxury_stock_reader`	Groupe	Lecture	Dashboard
`luxury_stock_app`	Connexion, membre de reader	Lecture	Connexion du dashboard
`luxury_stock_writer`	Groupe	Lecture, insertion, mise à jour	Ingestion et prévisions
`luxury_stock_pipeline`	Connexion, membre de writer	Lecture, insertion, mise à jour	Connexion du pipeline
`luxury_stock_maintainer`	Groupe	Lecture, mise à jour, suppression	Accordé temporairement à un compte nommé (rectification, effacement)

Les droits implicites du rôle `PUBLIC` sont retirés. Le chiffrement TLS, le chiffrement au repos, la réplication PostgreSQL et le gestionnaire de secrets sont des contrôles requis en production ; ils ne sont pas actifs dans l'environnement local.

7. Risques IA et contrôles

Le registre détaillé figure dans `ai_risk_register.xlsx` : 13 risques, évalués par une grille probabilité x impact dont le niveau est calculé par formule, avec statut de la mesure, preuve, risque résiduel et règle d'acceptation. Les principaux risques sont :

- fuite de données pendant l'évaluation ;
- données incorrectes ou incomplètes (seul risque de niveau critique) ;
- erreurs de prévision et dérive de la demande ;
- surconfiance dans une recommandation ;
- biais de disponibilité entre catégories et sous-estimation de la demande pendant les ruptures ;
- évaluation trop optimiste du modèle ;
- rupture de traçabilité ;
- exposition de secrets, accès non autorisé et dépendances vulnérables ;
- introduction de données personnelles réelles.

La matrice `controls.md` associe chaque contrôle aux risques qu'il couvre et indique son statut : actif, à activer ou planifié. Les contrôles incluent une séparation temporelle, une comparaison à une baseline, des tests, des métriques, des seuils de monitoring, une validation humaine et une procédure de réentraînement.

8. RGPD et conformité

Le MVP ne traite pas de données personnelles. Cette conclusion est limitée au périmètre synthétique fourni. Si des ventes associées à des personnes étaient ajoutées, il faudrait déterminer la base légale, documenter le registre des traitements, limiter la conservation, gérer les droits des personnes (procédure dans `data_subject_rights.md`), sécuriser les accès et réaliser une analyse d'impact si nécessaire.

Le projet reprend les principes de minimisation, sécurité, traçabilité et responsabilité. Les références et limites sont conservées dans la documentation plutôt que présentées comme une certification juridique.

9. Incidents et violations

En cas d'anomalie de données :

1. la ligne concernée est isolée en quarantaine et, si une règle `fail_on_*` est activée, le pipeline s'arrête ;
2. la raison est enregistrée ;
3. l'équipe technique analyse l'impact ;
4. le propriétaire métier valide la reprise ;
5. le rapport est conservé avec la version des données.

En cas d'exposition d'un secret : révocation immédiate, remplacement, analyse des logs et revue du dépôt. Le scan de secrets exécuté à chaque push (`secret-scan.yml`) sert de détection. En cas de données personnelles réelles, l'organisation compétente doit évaluer les obligations de notification.

10. Supervision et amélioration continue

Les rapports JSON du dossier `reports/` constituent les preuves de contrôle : qualité, transformation, pipeline, évaluation, explicabilité, monitoring, dérive, biais indicatif et scalabilité. Le registre des risques est revu chaque trimestre et à chaque réentraînement.

Un réentraînement est recommandé si :

- la MAE mesurée sur la fenêtre de test dépasse de plus de 10 % la MAE de référence (`monitoring.retrain_mae_increase_pct` dans `03_pipeline/config/config.yaml`) ;
- l'erreur dépasse les seuils définis dans `reports/monitoring_reference.json` ;
- la distribution des ventes dérive durablement (indice PSI de `drift_report.json`) ;
- une règle qualité critique est violée ;
- de nouvelles données ou catégories sont ajoutées.

11. Limites et références

- Le jeu est synthétique et les facteurs externes (promotions, événements, contraintes fournisseurs) ne sont pas représentés.
- La cible `net_units_sold` mesure des ventes, non la demande : pendant une rupture (`stockout_flag`), les ventes sous-estiment la demande.
- L'évaluation est à une semaine d'horizon : les retards de la période de test utilisent les valeurs observées, alors que les prévisions futures sont calculées de façon récursive sur 12 semaines.
- Elle repose sur une seule fenêtre de test de 12 semaines et sur une baseline en moyenne glissante de quatre semaines ; elle ne remplace pas un rétro-test sur plusieurs fenêtres.
- Le modèle ne remplace pas un système de planification industriel. La solution est conçue comme un MVP démontrable, reproductible et extensible.

Références à consulter

- RGPD, Règlement (UE) 2016/679 : https://eur-lex.europa.eu/eli/reg/2016/679/oj
- CNIL, principes de protection des données : https://www.cnil.fr/
- Loi Informatique et Libertés : https://www.legifrance.gouv.fr/loda/id/JORFTEXT000000886460/
- ISO/IEC 27001, système de management de la sécurité de l'information : https://www.iso.org/isoiec-27001-information-security.html
- ANSSI, recommandations de sécurité : https://cyber.gouv.fr/
- Référentiel général d'accessibilité pour les administrations, RGAA : https://accessibilite.numerique.gouv.fr/rgaa/

Ces références servent à justifier les principes retenus. Elles ne constituent pas une certification juridique, ISO ou ANSSI du MVP.

Historique du document

Version	Date	Modifications
0.1	Version initiale	Premier plan de gouvernance.
1.1	2026-10-15	Évaluation décrite comme une évaluation à une semaine d'horizon, chemin de configuration du seuil de réentraînement corrigé.
1.0	2026-10-14	Ordre des sections aligné sur la table des matières, lignage intégré avec texte alternatif, politique d'anomalies, rôles de base de données, nouveaux risques, seuil de réentraînement reformulé, limites complétées.